

Synthetic Image Detection

Antonio Mattesco

antonio.mattesco@studenti.unipd.it

Flavio Kaci

flavio.kaci@studenti.unipd.it

Anna Glado

anna.glado@studenti.unipd.it

Abstract

This report presents the work of our Vision and Cognitive System project, focused on the detection of synthetic images generated by diffusion models. Drawing inspiration from previous research, including the SuSy model, we evaluated various neural network architectures for this task, such as Convolutional Neural Networks (CNNs), ResNets, and SuSy itself. In our experiments, we explored the effects of preprocessing techniques, such as resizing and feature extraction, on the models' ability to detect synthetic images. Our findings indicate that the smash-and-reconstruction method did not provide any improvement in detection performance and may even be detrimental. On the other hand, the SuSy model demonstrated the highest performances in identifying synthetic images across different configurations.

We further assessed the generalization capabilities of the models by testing them on an external dataset, offering insights into their robustness. This report highlights key aspects of synthetic image detection, including the importance of architectural choices and preprocessing steps.

1. Introduction

The rise of generative models, particularly diffusion models, has significantly advanced the field of synthetic image generation. Unlike traditional image generation techniques, diffusion models iteratively transform a noise signal into a high-quality image, making them capable of producing highly realistic synthetic images. While this progress is a breakthrough for many applications, it also poses significant challenges in detecting such images, which can be used maliciously in areas like deepfakes, misinformation, and data manipulation.

Synthetic image detection has become a critical task to ensure the integrity of digital content. Traditional methods, such as those used for detecting images generated by Generative Adversarial Networks (GANs), have been widely explored. However, diffusion models present new chal-

lenges due to their superior image quality and the subtle differences they exhibit compared to real images. These challenges necessitate the development of more sophisticated detection techniques, particularly those leveraging advanced machine learning models.

In this paper, we focus on detecting synthetic images specifically generated by diffusion models, using state-of-the-art neural network architectures. We explore Convolutional Neural Networks (CNNs), ResNet models, and SuSy, a model from prior research, as potential tools for detection. We also examine the impact of preprocessing techniques and their role in enhancing detection performance.

Accurate detection of synthetic images is vital in many sectors, including social media, journalism, and cybersecurity. It can help prevent the spread of harmful or misleading content, ensuring the reliability and authenticity of digital media. This work provides an overview of the current methods for synthetic image detection, highlighting the importance of tackling these challenges in an era where synthetic images are increasingly indistinguishable from real ones.

2. Related Work

The detection of synthetic images has emerged as a crucial challenge due to the increasing sophistication of generative models, particularly Generative Adversarial Networks (GANs) and diffusion models. Early approaches to distinguishing synthetic images relied on traditional image processing techniques, such as statistical analysis and texture-based methods [4]. However, as the quality of generated images improved, these methods became less effective, necessitating the shift towards deep learning-based approaches.

Deep learning methods, particularly Convolutional Neural Networks (CNNs), have gained prominence in synthetic image detection. The introduction of GANs by Goodfellow et al. [5] marked a significant leap in the photorealism of synthetic images, posing new challenges for detection. To address this, several studies have utilized CNNs for detecting images generated by GANs [15]. These models excel at capturing subtle artifacts introduced during the genera-

tion process, which are difficult to detect using traditional methods.

In addition to CNNs, more advanced architectures, such as Residual Networks (ResNets) [6], have been leveraged for their ability to learn deep, residual connections that enhance feature extraction, crucial for distinguishing between real and synthetic images. These networks have proven effective in detecting inconsistencies that may arise in generated images, making them a popular choice for synthetic image detection [11].

The advent of diffusion models [7] has further complicated the detection landscape. These models generate high-quality images by gradually denoising random noise, presenting new challenges for traditional and deep learning-based detection methods. Recent research has focused on improving detection strategies for images produced by these models, incorporating advanced techniques such as metric learning and attention mechanisms to enhance detection performance [12].

Furthermore, some studies have explored the use of hybrid approaches that combine feature extraction methods with machine learning classifiers. Techniques like texture analysis and the "smash and reconstruction" approach have been used to identify synthetic images, but their effectiveness remains limited in real-world applications [9], as they often fail to generalize across different generative models [3].

In summary, while deep learning-based methods, particularly CNNs and ResNets, have shown significant promise in detecting synthetic images, challenges remain due to the continuous evolution of generative models. Future work will likely focus on improving the robustness and generalization of detection techniques, particularly for newer generative approaches such as diffusion models.

3. Dataset

In this study, we use the SuSy Dataset, which was introduced in [1]. This dataset is designed to evaluate models for synthetic image detection and consists of a collection of both authentic and synthetic images.

The authentic images are sourced from the widely used COCO dataset [10], which includes a diverse range of real-world images. The synthetic images in the SuSy dataset are generated using several state-of-the-art generative models, including DALLE3, StableDiffusion (SD1.X and SDXL), and Midjourney (MJ 1/2 and MJ 5/6). These models represent the forefront of image generation technology and provide a broad spectrum of synthetic image characteristics for testing detection models.

The dataset consists of a total of 25,561 images, which are split into training, validation, and test sets using a 60%-20%-20% random split.

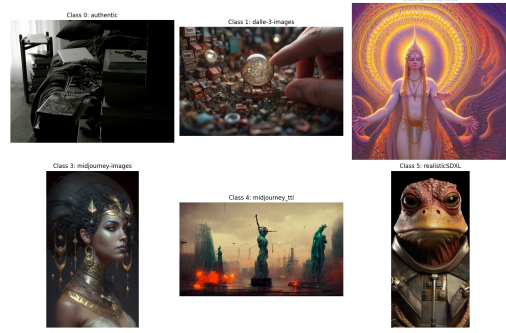


Figure 1. Comparison of images generated by different generative models.

The dataset includes six classes: class 0 represents authentic images, while classes 1 through 5 correspond to synthetic images generated by different models. For the binary classification task, where real and fake images are distinguished, a portion of the images were sampled to ensure a balanced representation of both real and fake images, while also considering the limited resources available.

After the split, the training set contains 2,967 authentic images and 2,295 fake images, for a total of 5,262 images. The validation and test sets each contain 2,096 images.

This balanced dataset is then used to train a binary classifier capable of effectively distinguishing between real and fake images, enabling models to be trained within a limited amount of time. Google Colab (free version) with an Intel Xeon CPU @ 2.20GHz (2 vCPUs, 1 core) and 12GB RAM was used for training.

4. Methodology

Following the dataset balancing, several preprocessing steps are performed to further refine the data for model training. These steps include resizing the images to a uniform resolution and applying data transformations, ensuring that the data is consistent and ready for effective training. Additionally, techniques such as *Smash&Reconstruction* [16] are explored as part of the preprocessing process.

4.1. Image Resizing

The images in the dataset have different dimensions, which can pose a challenge for training a convolutional neural network (CNN), as the architecture requires input images of the same size. To address this, we resize all images to a consistent size of 256x256 pixels, ensuring compatibility with the CNN model.

We achieve the resizing using bicubic interpolation [8], which provides a good balance between performance and image quality. An example of an original image and its resized counterpart is shown below.



Figure 2. Comparison between the original and resized image

4.2. Smash & Reconstruction Method

The Smash and Reconstruction (S&R) process is applied to image patches to distinguish rich textures from poor textures in the images. The primary objective of this technique is to explore how real and fake images differ in terms of texture complexity.

The process begins by dividing the image into patches of a predetermined size (32x32 pixels). For each patch, we compute the pixel gradient, which involves finding the differences in pixel values along horizontal, diagonal, and counter-diagonal directions. The gradients are then aggregated to categorize the patches into two groups: rich texture patches and poor texture patches. Rich texture patches exhibit greater complexity and detail, while poor texture patches show simpler and less intricate patterns, this is because patches with rich texture will have higher pixel gradient values when compared to poor texture patches.

The process should extract a first fingerprint from the image, and this fingerprint, it is hoped, changes when comparing real and fake images. While rich textures appear to maintain the same level of complexity for both real and fake images, real images tend to exhibit more complex poor textures compared to those of fake images, as shown in Figure 3.

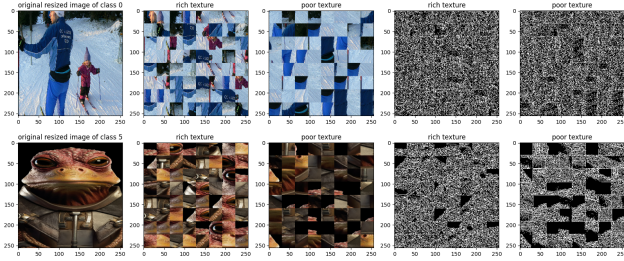


Figure 3. Smash and Reconstruction process for a real image (top) and an AI-generated image (bottom).

This method emphasizes texture features that are crucial for distinguishing between real and fake images. Subsequently, both rich and poor textures are processed through separate convolutional layers, followed by a batch normalization layer and a HardTanh activation function. This se-

quence helps to extract distinct fingerprints for both rich and poor textures. Finally, the elementwise difference between the two feature maps is computed, which is then fed into a classifier. While the initial steps are deterministic and identical for each image, the convolutional layers require training and are integral to the model.

4.3. Data Preparation for Training

At this stage, we ensure that the data is properly structured for training the models. The primary goal is to assign a target label to the images, where real images are assigned a label of 0, and AI-generated images are labeled as 1. Additionally, we compare the performance of similar architectures trained on two types of data: one that has undergone the Smash and Reconstruction (S&R) preprocessing and one that has not. This comparison allows us to evaluate the impact of the S&R technique on model performance and determine whether it enhances the model’s ability to classify real versus AI-generated images.

For the images that have been preprocessed with Smash and Reconstruction, we structure the dataset by extracting the rich and poor texture patches. These patches are used as input features for the models. On the other hand, we also prepare a version of the dataset without S&R preprocessing. For this dataset, we apply a scaling function to standardize the features. Specifically, the MaxMin Scaler is used, which normalizes the feature values by rescaling them to a fixed range, between 0 and 1, ensuring that all features contribute equally to the model. Scaling the features is important because when the features have different scales, it can lead to optimization challenges during training. Specifically, having features with different scales can result in an irregular loss function surface, which makes it difficult for optimization methods like gradient descent to converge. If the features are not properly scaled, the optimizer may struggle to find the optimal direction, and the gradients may become too large for large-scale features or too small for smaller-scale features, causing gradient saturation or instability.

In addition to improving optimization, scaling helps mitigate overfitting. Without proper scaling, features with large scales would dominate the learning process, and smaller features could be ignored by the model. By preparing these two different datasets, we can compare how preprocessing techniques like Smash and Reconstruction and scaling affect the performance of the model. This enables us to make data-driven decisions on which preprocessing methods lead to the best model performance.

5. Experiments

This section presents the various architectures tested for the real-fake image detection task, which will also be tested and compared with SuSy. Specifically, a simple CNN architecture and a ResNet-18 model with an additional output

layer were evaluated. Both models were trained on resized images as well as on rich-poor texture features.

5.1. CNN Architecture

The convolutional neural network (CNN) model consists of two main components: a feature extractor based on convolutional layers and a fully connected classifier (MLP). For simplicity a graphical visualization is shown in 4.

The feature extractor includes a series of five convolutional layers. The first convolutional layer uses a 5x5 kernel with 3 input channels (RGB images) and 6 output channels, followed by a max pooling operation with a 2x2 window to reduce dimensionality. A second convolutional layer with 8 output channels and a 3x3 kernel is applied, followed by another max pooling operation. A batch normalization operation is used to stabilize training and prevent overfitting. This process continues with additional convolutional layers, progressively increasing the number of output channels and applying pooling operations to reduce spatial resolution.

The convolutional layers are followed by a block of fully connected layers (MLP). This block starts with a dropout layer (with a probability of 0.5) to prevent overfitting, followed by two fully connected layers, ending up with a model that has a total of 825,714 trainable parameters.

For comparison of the effectiveness of the Smash and Reconstruction (S&R) technique, the same architecture is applied to inputs consisting of rich and poor textures. However, before passing the textures through the main model, they are preprocessed by a feature extraction module. This module consists of a convolutional layer with a 3x3 kernel and 3 output channels, followed by batch normalization and a Hardtanh activation function. The feature extraction module add some trainable parameters to a total of 825,786.

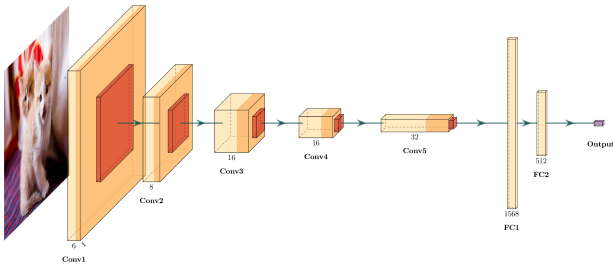


Figure 4. CNN architecture, where dark orange layers represent max pooling and ReLU activation components, while light orange layers correspond to convolutional layers (Conv) and fully connected (FC).

5.2. ResNet18

ResNet18, short for Residual Network with 18 layers, is a deep convolutional neural network known for its use of residual connections, which allow for more efficient training of deeper networks by alleviating the vanishing gradient

problem. Unlike traditional CNN architectures, ResNet18 incorporates skip connections that enable the model to learn residual mappings, enhancing its performance on complex tasks. We choose ResNet18 because it inspires also the SuSy model, making it easier to transition from a simple CNN architecture to a deeper model like SuSy.

For the task of synthetic image detection, we adapted the ResNet18 model to fit the requirements of our dataset. The original ResNet18 model has an higher dimensional output layer. However, since our dataset involves only two classes—real and fake images—we add to the original output layer a new one that consists of just two neurons.

As was done for CNNs, for the ResNet18 applied to S&R images, two distinct convolutional layers are added at the top. This ensures that after extracting fingerprint features from both rich and poor textures and computing their element-wise difference, the resulting feature map is used as input to the model. For both models, the parameters of the ResNet backbone were frozen, and only the newly added layers were trained.

5.3. SuSy Model

The architecture consists of two main modules: a feature extractor and a multi-layer perceptron (MLP), designed for efficiency. The model, named SuSy, has a total of 12.7 million parameters, with the feature extractor contributing 12.5 million parameters and the MLP accounting for the remaining 197K parameters.

The convolutional feature extractor follows a ResNet-18-based scheme and consists of five stages. The output from each block is fed into multiple bottleneck modules, arranged in a staircase pattern. Each bottleneck module consists of three 2D convolutional layers. The bottleneck modules operate at different stages of the network, with each level of bottlenecks receiving input from the current stage and, except for the first bottleneck in each level, from the previous bottleneck module.

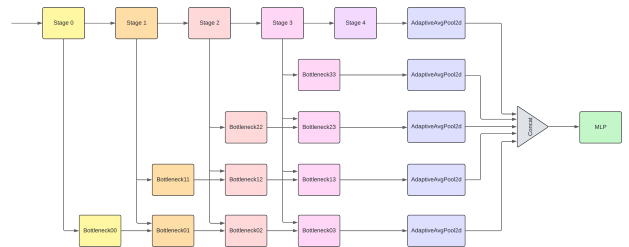


Figure 5. SuSy model architecture

The outputs of each bottleneck level, along with the output from stage 4, are passed through a 2D adaptive average pooling layer, and then concatenated to form the feature map that feeds into the MLP. The MLP consists of three

fully connected layers with 512, 256, and 256 units, respectively. Dropout (with a rate of 0.5) is applied between layers to mitigate overfitting. The output layer of the MLP has six units, corresponding to the six classes in the dataset (five synthetic models and one real image class).

The model can function as a detector by either selecting the class with the highest probability or by summing the probabilities of the synthetic classes and comparing them to the real class. Additionally, the model can serve as a recognition model by selecting the class with the highest probability as the output.

6. Results

Model	Val Accuracy	Test Accuracy	Recall (Class 1)
CNN	81.39%	81.39%	0.79
CNN S&R	50.91%	57.39%	0.43
ResNet18	89.07%	86.00%	0.71
ResNet18 S&R	63.50%	64.00%	0.51
SuSy	-	98.00%	1.00

Table 1. Performance comparison of the evaluated models on the validation set and test set.

Table 1 presents the performance comparison of the evaluated models on the test set. Since the dataset is balanced, accuracy serves as a meaningful metric to assess overall performance. However, the primary focus of this study is to maximize the recall of class 1 (synthetic images), ensuring that fewer fake images are misclassified as real.

The SuSy model, which was externally imported, achieved the highest recall, detecting all synthetic images. Among the trained models, ResNet18 exhibited the best trade-off between accuracy and recall, while the Smash and Reconstruction (S&R) preprocessing negatively impacted performance across all architectures. This suggests that the reconstruction step may introduce artifacts that hinder the models’ ability to differentiate between real and fake images.

As expected, deeper and more computationally expensive architectures, such as ResNet18, achieved better performance in terms of balancing accuracy and recall. This indicates that more complex models are likely to be more effective at distinguishing between the two classes, but they come at the cost of increased computational resources. Additionally, further improvement could be achieved by increasing the number of synthetic images in the dataset, allowing the models to specialize more in recognizing fake images and improving their recall for class 1.

6.1. Challenges in Poor Texture-Based Classification

A brief digression is necessary to analyze certain steps and potential improvements for classification based on the

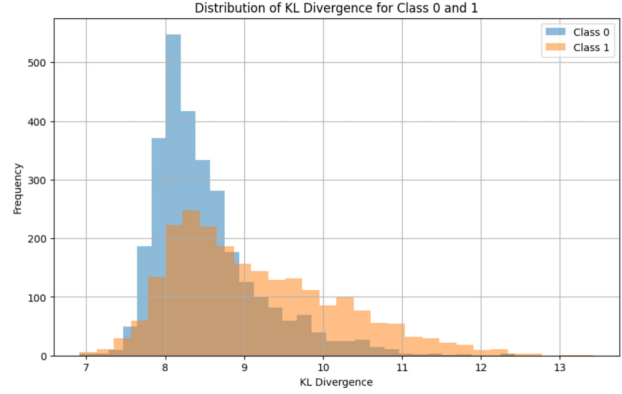


Figure 6. KL-Divergence distribution for poor textures maps

S&R technique. As stated in [16], poor textures exhibit higher discriminative power. However, analyzing the KL divergence (Figure 6) of poor textures for real (class 0) and fake (class 1) images reveals that their distributions are quite similar, sharing the same mode but differing significantly in the right tail. This suggests potential challenges in the predictive capabilities of the associated models.

Following this observation, an adapted model—whose architecture summary is shown in 7—was trained exclusively on poor textures using a subset of approximately 500 images. The resulting accuracy on the test set reached 0.74. This analysis is not further extended here, but it is included to provide insights for potential future developments.

Layer (type:depth-idx)	Output Shape	Param #
ImprovedClassifier	[32, 2]	—
Conv2d: 1-1	[32, 32, 256, 256]	320
ReLU: 1-2	[32, 32, 256, 256]	—
Conv2d: 1-3	[32, 64, 256, 256]	18,496
ReLU: 1-4	[32, 64, 256, 256]	—
Dropout: 1-5	[32, 64, 128, 128]	—
Conv2d: 1-6	[32, 128, 128, 128]	73,856
ReLU: 1-7	[32, 128, 128, 128]	—
Conv2d: 1-8	[32, 128, 128, 128]	147,584
ReLU: 1-9	[32, 128, 128, 128]	—
Dropout: 1-10	[32, 128, 64, 64]	—
Conv2d: 1-11	[32, 256, 64, 64]	295,168
ReLU: 1-12	[32, 256, 64, 64]	—
Conv2d: 1-13	[32, 256, 64, 64]	590,080
ReLU: 1-14	[32, 256, 64, 64]	—
Dropout: 1-15	[32, 256, 32, 32]	—
AdaptiveAvgPool2d: 1-16	[32, 256, 1, 1]	—
Linear: 1-17	[32, 2]	514
Total params: 1,126,018		
Trainable params: 1,126,018		
Non-trainable params: 0		
Total mult-adds (Units.GIGABYTES): 271.59		

Figure 7. Model architecture for poor textures

7. Experimental Dataset

In this section, we assess the performance of our models by applying them to test images from an external dataset. The dataset is carefully selected to evaluate how well the models generalize to new, unseen images.

7.1. Dataset Description

The external dataset [13, 14] consists of 20000 train images, from which we sample 1000 real and 1000 fake images. All images are sampled to ensure they meet the necessary resolution requirements for the SuSy model, that do not accept images with height or width lower than 224. The images are then preprocessed for each model according to 4. Some samples from both the real and fake image categories are shown in Figure 8.

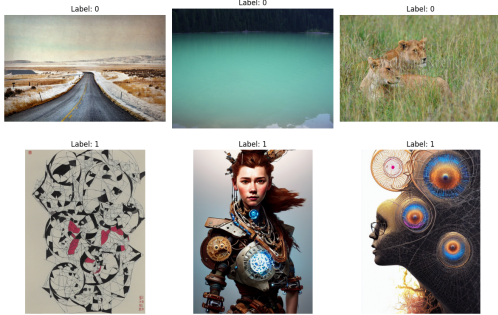


Figure 8. Sample images from the external dataset. Real images are labeled as 0 (first row), while fake images are labeled as 1 (last row).

7.2. Results

The results of the model evaluations are presented in Table 2, where we compare the accuracy of each model on the external dataset. SuSy achieves the highest accuracy, making it the best-performing model. Notably, models incorporating S&R perform significantly worse, with accuracies below 50%.

Model	Accuracy	Recall (Class 1)
ResNet18 with S&R	39.90%	0.28
CNN with S&R	42.40%	0.32
CNN	76.70%	0.71
ResNet18	78.55%	0.75
SuSy	90.10%	0.99

Table 2. Accuracy results for different models on the external test dataset.

Additionally, as reported in [1], a study involving 10 volunteers aged 22-30, all active social media users likely exposed to digital content, was conducted. The evaluation utilized the In-the-wild dataset, which includes both authentic and synthetic images (see §3.3 in [1]). Table 3 presents a comparison of the recall performance of SuSy at both patch and image levels against human evaluators.

Assuming the dataset has not similar images as those used in the study (look the SuSy performances), we can assert that models trained on the original data and with the

	Authentic	Synthetic
Avg. Human Evaluator	72.82%	69.14%
SuSy (Patch)	72.56%	64.24%
SuSy (Image)	75.21%	69.70%

Table 3. In-the-wild recall performance of SuSy and human evaluators, following [1].

S&R technique will be probably outperformed by a human classifier.

8. Conclusions and Future Work

In this work, we investigated various architectures for synthetic image detection, comparing a custom CNN model, ResNet18, and the SuSy model. Our results demonstrate that more complex models, such as ResNet18 and SuSy, offer superior discriminative power compared to simpler architectures. On the other hand, the Smash & Reconstruction technique performed worse than traditional methods, suggesting that reconstructing textural details may introduce noise rather than enhancing the separation between real and generated images.

Through our experiments, we learned that certain methods, like Smash & Reconstruction, may not always contribute positively and could introduce unnecessary complexity or noise into the model, reducing its overall efficacy. This highlights the importance of carefully selecting techniques that align with the problem at hand.

Tests on an external dataset confirmed the generalization capability of the SuSy model, while other architectures experienced a performance drop, emphasizing the need for further optimization. Additionally, comparisons with human evaluators suggest that current models, while effective, could benefit from a more adaptive approach to enhance robustness on unseen images.

In future work, it would be valuable to delve further into the findings discussed in 6.1. Furthermore, increasing the amount of data—particularly by integrating more examples of fake images—could significantly improve model performance, especially by increasing the recall for class 1 (synthetic images). This would enhance the model’s ability to distinguish fake images more effectively and reduce the likelihood of misclassification.

A promising direction for future research could involve developing more complex architectures, leveraging advanced techniques like the method proposed in [2], which combines the original image with the decoded noise map and reconstruction obtained from Stable Diffusion. This approach may push the boundaries of synthetic image detection but require further experimentation, more data, and increased computational power.

References

- [1] Pablo Bernabeu-Perez, Enrique Lopez-Cuena, and Dario Garcia-Gasulla. Present and future generalization of synthetic image detectors. *arXiv preprint arXiv:2409.14128*, 2024.
- [2] George Cazenavette, Avneesh Sud, Thomas Leung, and Ben Usman. Fakeinversion: Learning to detect images from unseen text-to-image models by inverting stable diffusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10759–10769, 2024.
- [3] Jiajun Chen, Xian Wang, and Xin Zhang. Limitations of current synthetic image detection methods: Challenges and future directions. *IEEE Transactions on Neural Networks and Learning Systems*, 32(7):2154–2165, 2021.
- [4] Ziv Farbman, Ron Kimmel, and Guillermo Sapiro. Edge detection via scale-space analysis of multiresolution histograms. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 30(2):375–387, 2008.
- [5] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, volume 27, 2014.
- [6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [7] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *arXiv preprint arXiv:2006.11239*, 2020.
- [8] R. Keys. Cubic convolution interpolation for digital image processing. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 29(6):1153–1160, 1981.
- [9] Yu Li, Hongyu Zhang, and Zhiqiang Chen. Detecting gan-generated images using feature-based models. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1133–1141, 2019.
- [10] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014.
- [11] Alvaro Perez and Elena Garcia. Resnet-based architectures for image classification: A review. *Neural Computing and Applications*, 33(5):1419–1433, 2021.
- [12] Lin Yang and Wei Chen. Diffusion models for synthetic image detection: A deep learning approach. *Journal of Machine Learning Research*, 23(2):1–14, 2022.
- [13] Chunli Zhang Chengyue Wu Xihui Liu Lei Bai Wanli Ouyang Zeyu Lu, Di Huang. Sentry-image leaderboard. <https://github.com/Inf-imagine/Sentry>, 2023.
- [14] Lei Bai Jingjing Qu Chengyue Wu Xihui Liu Wanli Ouyang Zeyu Lu, Di Huang. Seeing is not always believing: Benchmarking human and model perception of ai-generated images, 2023.
- [15] Zhiwei Zhang, Jian Zhu, Wei Yi, and Wen Xu. Detecting synthetic images from gans using deep neural networks. *IEEE Transactions on Image Processing*, 28(10):5044–5055, 2019.
- [16] Nan Zhong, Yiran Xu, Zhenxing Qian, and Xinpeng Zhang. Rich and poor texture contrast: A simple yet effective approach for ai-generated image detection. *arXiv preprint arXiv:2311.12397*, 2023.